

# Mathematical foundations and theoretical argument

## 数学基础与理论论证

### ABSTRACT / 摘要

本章建立 Cantilune 关于 Agent Orchestration 的完整数学理论。首先，协调系统被建模为一个由签名约束、通过协调事件不断改变的协调世界，从而明确整个理论所处理的状态、事件与规则；随后引入协调语义，规定这些事件如何成为合法、可观察且可重放的真实执行；在此基础上，同一次协调发生被分别投影到 DAG、Petri、 $\pi$  与态射四个观测域，以解释其对依赖结构、资源流动、交互行为和组合形式的不同影响；进一步通过路径保持、反向解释、终态一致、签名重索引与发生身份保持，证明这些观测在不同执行阶段和不同 epoch 中仍然描述同一条协调历史；最后，利用有限高度证据序和由原生事件支撑的有限 Markov kernel，证明协调执行在明确的稳定窗口、公平性与正进展条件下几乎必然进入并永久保持在稳定区域。由此，本章形成一条从协调世界的构造、协调变化的发生、多维语义的解释和一致性，到长期反馈稳定化的完整数学闭环。

### 3.1 The Coordination World: States, Events, and Rules / 协调世界：状态、事件与规则

本节给出 Cantilune 对 Agent Orchestration 的顶层数学建模。其将协作理解为一个具有明确结构、资源归属、证据与策略状态的协调世界。该世界在每一时刻表现为配置  $X_i \in \text{Config}(\Sigma)$ ，并通过可识别、可审计且可重放的协调事件  $e$  演化为后继配置  $X_{i+1}$ 。为说明何种协作结构能够被构造、何种结构变化能够发生，本节进一步从签名  $\Sigma$  引入协调演算  $\text{Calc}_\Sigma = (\mathcal{C}_\Sigma, \mathcal{R}_\Sigma)$ ：其中  $\mathcal{C}_\Sigma$  给出类型正确的静态协作结构语言， $\mathcal{R}_\Sigma$  给出其允许的重写动力学。由此，状态、事件、结构与演化被置于同一形式框架中，并为后续的 DPO 重写、四投影、replay 与终态分析建立共同的数学起点。

#### 3.1.1 Coordinative World State and Events / 协调世界与协调事件

Agent Orchestration 通常被建模为多个 agent 之间互相通信，但是对于真实协作而言，orchestration 不止互通这么简单，其还需要考虑任务归属、执行权限与预算、结果可预期与可重放等等。

因此，Cantilune 将 Agent Orchestration 建模为“一个不断被合法改变的协作世界”。该世界每时刻都具有一个状态，被定义为“协调世界状态 ( $X$ )”。在给定当前签名  $\Sigma$  时， $X$  是一个合法运行时配置：

$$X \in \text{Config}(\Sigma). \quad (3.1)$$

它包含当前状态下的参与者、工作对象、工作关系、稀缺资源、收集到的证据、适用策略等等。该状态会不断发生变化，变化由事件产生，该事件被定义为“协调事件 ( $e$ )”。因此，整个 orchestration 的过程可以被定义为：

$$\boxed{X \xrightarrow{e} X'}. \quad (3.2)$$

**Example / 案例**

让 planner 将任务  $T$  委派给 coder, 它是一个事件  $e_{\text{delegate}}$ , 使世界从  $X$  变成  $X'$ 。在  $X$  中, 任务  $T$  由 planner 负责; 在  $X'$  中, 任务  $T$  由 coder 负责或由 coder 执行。  
若任务需要唯一写权限, 则该权限从 planner 转移给 coder; planner 与 coder 之间可能产生一个新的私有会话; reviewer 的依赖 状态变成“等待 coder 的候选结果”; 事件记录里保留任务  $T$ 、双方身份、权限 token、会话名和授权证据。

这就是 Cantilune 的顶层建模: **orchestration** 是对协作世界的受约束、可识别、可重放的状态变换。

**3.1.2 Composition of Coordinative World / 协调世界的构成**

我们用  $\Sigma$  定义协调世界中允许出现的类型, 作为“协作语言与类型合同”, 公式表示为:

$$\boxed{\Sigma = (\text{Obj}, \text{Gen}, \text{in}, \text{out}, \text{mode}, \text{contract})} \quad (3.3)$$

其中:

1. Obj: 可出现的东西的类别, 例如 Task、Artifact, 以及写权限、会话和审核类型。
2. Gen: 允许出现的操作类别, 例如 delegate、accept、reject、createSession。
3. in 与 out: 每一种操作需要什么、产生什么。
4. contract: 端口或操作需要满足的合同。
5. mode: 是签名中每种类型的结构使用方式, 即该类型的值能否被复制、能否被丢弃, 回答的不是“端口是输入还是输出”, 而是一份这种类型的东西, 是否能被复制, 是否能被不用就扔掉。

**Static Coordination Calculus and Rewrite Foundations / 静态协调演算与重写基础**

定义:

$$\text{Calc}_\Sigma = (\mathcal{C}_\Sigma, \mathcal{R}_\Sigma) \quad (3.4)$$

其中  $\mathcal{C}_\Sigma$  表示在签名  $\Sigma$  下所有合法组合结构,  $\mathcal{R}_\Sigma$  表示允许改变这些结构的协调规则。  
更具体的,  $\mathcal{C}_\Sigma$  表示在签名  $\Sigma$  被表示为:

$$\mathcal{C}_\Sigma = \text{FreeSMC}(\Sigma) / \equiv_\Sigma \quad (3.5)$$

自由 SMC 提供组合语义, 表示所有“不违反组合规则”的结构;  $\equiv_\Sigma$  则表示取商, 即因为 FreeSMC 生成的东西太多, 所以通过取商把等价结构合并。

协调规则进一步表示为:

$$\boxed{r \in \mathcal{R}_\Sigma, r = (L \leftarrow K \rightarrow R)} \quad (3.6)$$

其中  $L$  表示匹配前结构,  $K$  表示接口保持部分,  $R$  表示替换后结构。该式表示什么被找到、什么被保留、什么被替换, 从而保证了信息在变化过程中不会丢失。

另外有三个性质, 同样为后面的“状态”和“事件”提供生成基础:

**Free SMC 泛性质** 令  $F_\Sigma : \Sigma \rightarrow \mathcal{C}_\Sigma$  为签名到自由对称么半范畴的规范嵌入。对于任意对称么半范畴  $\mathcal{D}$  及签名解释  $f : \Sigma \rightarrow \mathcal{D}$ ，存在一个强对称么半函子  $\bar{f} : \mathcal{C}_\Sigma \rightarrow \mathcal{D}$ ，使  $\bar{f} \circ F_\Sigma \cong f$ ；该函子在相干么半自然同构意义下唯一。这表明，一旦基本组件的解释被确定，复杂组合结构的解释便由其组合规律确定，而不能任意指定。

**DPO 结果唯一性** 如果  $X \xrightarrow{r} Y_1$  以及  $X \xrightarrow{r} Y_2$ ，则  $Y_1 \cong Y_2$ ，这表示同一个协调规则、同一个匹配，不会产生两个语义冲突结果。

**并发交换** 若  $r_1 \parallel r_2$  满足独立性  $Independent(r_1, r_2)$ ，则  $X \xrightarrow{r_1} X_1 \xrightarrow{r_2} Y$ ，且  $X \xrightarrow{r_2} X_2 \xrightarrow{r_1} Y'$ ，则满足  $Y \cong Y'$ ，这表示多个协调事件可以并行执行，并行执行的事件互不影响。

进一步的，我们定义  $\text{Config}(\Sigma)$ ：

$$\text{Config}(\Sigma) := \left\{ (v, V, E, \ell, D, U, N, o_D, o_U, o_N, O, \pi, Z) \mid \begin{array}{l} v \in \mathbb{N}, V \in \mathcal{P}_{\text{fin}}(\text{NodeId}), E \in \mathcal{P}_{\text{fin}}(\text{NodeId} \times \text{NodeId}), \\ \ell \in (\text{NodeId} \rightarrow \text{Gen}(\Sigma)), D \in \mathcal{P}_{\text{fin}}(\text{DataId}), \\ U \in \mathcal{P}_{\text{fin}}(\text{ResourceId}), N \in \mathcal{P}_{\text{fin}}(\text{NameId}), \\ o_D \in (\text{DataId} \rightarrow \text{NodeId}), \\ o_U \in (\text{ResourceId} \rightarrow \text{NodeId}), \\ o_N \in (\text{NameId} \rightarrow \text{NodeId}), \\ O \in \text{Observation}^*, \quad \pi \in \text{PolicyState}, \\ Z \in \mathcal{P}_{\text{fin}}(\text{TombstoneId}) \end{array} \right\}. \quad (3.7)$$

进而  $X$  可以表示为：

$$X = (v, V, E, \ell, D, U, N, o_D, o_U, o_N, O, \pi, Z) \in \text{Config}(\Sigma) \quad (3.8)$$

Table / 表 3.1: 协调世界配置的字段说明。

| 字段     | 类型                                                             | 形式含义             | 在 Agent Orchestration 中的含义                                              |
|--------|----------------------------------------------------------------|------------------|-------------------------------------------------------------------------|
| $v$    | $\mathbb{N}$                                                   | 配置记录的签名版本        | 当前世界所属的版本或 epoch 标识。它用于审计与 replay；它不是签名 $\Sigma$ 本身。                    |
| $V$    | $\mathcal{P}_{\text{fin}}(\text{NodeId})$                      | 活跃节点的有限集合        | 当前存在的 Agent、任务、工具调用、产物处理节点、审核节点等实例。                                     |
| $E$    | $\mathcal{P}_{\text{fin}}(\text{NodeId} \times \text{NodeId})$ | 活跃节点间的有限有向边集合    | 当前的依赖、控制、数据流或协作关系。例如“Reviewer 等待 Coder 的候选结果”。                          |
| $\ell$ | $\text{NodeId} \rightarrow \text{Gen}(\Sigma)$                 | 从节点标识到签名生成元的部分标注 | 指出某个节点在当前运行中是什么合法操作实例，例如 Planner、Coder、delegate、review 或 createSession。 |
| $D$    | $\mathcal{P}_{\text{fin}}(\text{DataId})$                      | 活跃数据 token 的有限集合 | 任务描述、计划、候选产物、测试结果、审核结论等可传递或可消费的工作对象。                                    |

Continued on next page

Table / 表 3.1: 协调世界配置的字段说明。(Continued)

| 字段    | 类型                                             | 形式含义             | 在 Agent Orchestration 中的含义                      |
|-------|------------------------------------------------|------------------|-------------------------------------------------|
| $U$   | $\mathcal{P}_{\text{fin}}(\text{ResourceId})$  | 活跃资源 token 的有限集合 | 不可随意复制的写权限、预算、配额、独占工具槽、锁等线性能力。                  |
| $N$   | $\mathcal{P}_{\text{fin}}(\text{NameId})$      | 活跃名字或会话的有限集合     | 私有 channel、任务 session、临时关联标识、可被传递的通信端点。         |
| $o_D$ | $\text{DataId} \rightarrow \text{NodeId}$      | 数据 token 的归属映射   | 哪个活跃节点当前负责、持有或有权提交某份产物。                         |
| $o_U$ | $\text{ResourceId} \rightarrow \text{NodeId}$  | 资源 token 的归属映射   | 谁当前持有唯一写权限、预算或独占执行资格。它保证资源转移不是悄悄复制。             |
| $o_N$ | $\text{NameId} \rightarrow \text{NodeId}$      | 名字或会话的归属映射       | 哪个节点当前控制、创建或负责关闭某个私有会话。                         |
| $O$   | Observation*                                   | 外部观察构成的有限序列      | 人类批准、评审意见、工具回执、环境输入、投票等。使用序列而非集合，因此发生顺序保留。      |
| $\pi$ | PolicyState                                    | 策略状态             | 当前授权模式、投票或 quorum 进度、重试策略、反馈聚合、epoch 内排名信息等。    |
| $Z$   | $\mathcal{P}_{\text{fin}}(\text{TombstoneId})$ | 已删除实体的墓碑集合       | 被删除的节点、资源或会话不会从审计历史中“失忆”；墓碑支持 trace 对账与 replay。 |

### 3.1.3 Composition of Coordinative Events / 协调事件的构成

与协调世界类似，我们也可以用一组变量来定义协调事件：

$$e = (v, \rho, X, X', m, \kappa, \nu, \alpha, \omega, k). \quad (3.9)$$

Table / 表 3.2: 协调事件的字段说明。

| 字段       | 回答的问题                     |
|----------|---------------------------|
| $v$      | 此次事件发生在哪个版本或 epoch?       |
| $\rho$   | 使用了哪条规则? 例如委派。            |
| $X$      | 事件发生前, 世界是什么状态?           |
| $X'$     | 事件发生后, 世界是什么状态?           |
| $m$      | 规则具体匹配到了 $X$ 中的哪些实体?      |
| $\kappa$ | 改变局部结构时, 哪些外部连接被保留?       |
| $\nu$    | 是否生成了新鲜名字, 例如新私有 session? |
| $\alpha$ | 什么授权、策略或资源证据允许这次事件?       |
| $\omega$ | 有什么外部观察或批准证据?             |
| $k$      | 这是内部、外部还是行政性事件?           |

其中尤其要注意的是  $e$  和  $\rho$  的区别，前者是一次具体发生的事件，后者则是“规则”，代表一种可发生变化的模板；同一条规则可以被多次、在不同对象上使用。

**Example / 案例**

假设当前配置中有：任务  $T$ ；Planner  $p$ ；Coder  $c$ ；唯一写权限  $w$ ；尚不存在的私有会话。  
规则只说：

$$\rho_{\text{delegate}} = \text{“允许委派任务”}. \quad (3.10)$$

实际事件则会说：

$$e_1 = (v, \rho_{\text{delegate}}, X, X', m_1, \kappa_1, \{s\}, \alpha_1, \omega_1, k). \quad (3.11)$$

其中：

- $m_1$  指明：本次匹配的是任务  $T$ 、Planner  $p$ 、Coder  $c$  与权限  $w$ 。
- $s$  指明：本次创建的新私有会话叫  $s$ 。
- $\alpha_1$  指明：Planner 确实有权把该任务和权限交给 Coder
- $\omega_1$  指明：若需要人工批准或评审记录，它是什么。
- $X'$  指明：任务责任、写权限归属、会话集合和 Reviewer 的等待关系具体变成什么。

如果 Planner 把另一个任务  $T'$  委派给另一个 Coder  $c'$ ，即使仍使用同一条规则，也已经是另一件事：  $e_2 \neq e_1$ ，因为  $m_2 \neq m_1$ 。

## 3.2 From Description to Execution: The Semantics of Coordination / 从描述到执行：协调语义

协调语义以良构协调状态为状态空间，以经重放验证的协调事件为转移标签，以原生合法转移关系连接连续状态，并由此生成有限或无限的协调执行路径。状态表示等价保证执行意义不依赖具体表示，事件可观察性从原生执行中选择对外可见的协调步骤，终态分类则解释一条可观察路径为何停止。因此，Agent Orchestration 的生命过程被表示为一条从初始协调状态出发、经连续合法事件演化并最终进入成功、外部等待或死锁状态的协调执行路径。

### 3.2.1 The Gap Between Describability and Executability / 可描述性与可执行性之间的语义缺口

在上一章，我们定义了协调世界  $X$  和协调事件  $e$ ，用于描述 Agent Orchestration 的整个生命过程；同时使用  $\Sigma$  为协调语言定义静态规范，为描述 Agent Orchestration 定义了一套语言标准。

但是，整个生命过程是如何发生的，到底生命过程中能发生什么样的变化，以及什么样的变化是符合规范的，这些问题却并没有被正确建模和描述清楚。因此，本章我们引入“协调语义 (The Semantics of Coordination)”来解决上述问题。

### 3.2.2 The Structure and Scope of Coordination Semantics / 协调语义的整体结构与作用范围

#### 3.2.2.1 The Structure of Coordination Semantics / 协调语义的整体结构

在考虑不同协调执行环境的背景下，协调语义被定义为：

$$\text{Sem}_{\Sigma, \mathfrak{e}} = (\text{St}_{\Sigma}, \text{Ev}_{\Sigma, \mathfrak{e}}, \Rightarrow_{\Sigma, \mathfrak{e}}, \text{Replay}_{\Sigma, \mathfrak{e}}, \simeq_{\Sigma, \mathfrak{e}}, \text{Visible}_{\Sigma, \mathfrak{e}}, \text{Outcome}_{\Sigma, \mathfrak{e}}) \quad (3.12)$$

其中，各数学对象的表述如下。

Table / 表 3.3: 协调语义的组成对象及其名称。

| 数学对象                                    | 中文名称         | 英文名称                                          |
|-----------------------------------------|--------------|-----------------------------------------------|
| $\text{St}_{\Sigma}$                    | 良构协调状态空间     | well-formed coordination-state space          |
| $\text{Ev}_{\Sigma, \mathfrak{e}}$      | 经重放验证的协调事件空间 | replay-verified coordination-event space      |
| $\Rightarrow_{\Sigma, \mathfrak{e}}$    | 原生合法协调转移关系   | native legal coordination-transition relation |
| $\text{Replay}_{\Sigma, \mathfrak{e}}$  | 去端点确定性重放核    | endpoint-free deterministic replay kernel     |
| $\simeq_{\Sigma, \mathfrak{e}}$         | 状态表示等价关系     | state-representation equivalence relation     |
| $\text{Visible}_{\Sigma, \mathfrak{e}}$ | 事件可观察性判定     | event-visibility predicate                    |
| $\text{Outcome}_{\Sigma, \mathfrak{e}}$ | 派生终态结果分类     | derived terminal-outcome classification       |

**良构协调状态空间  $\text{St}_{\Sigma}$**  良构协调状态空间表示所有合法协调世界状态构成的空间，写作：

$$\text{St}_{\Sigma} = \{X \in \text{Config}(\Sigma) \mid \text{WF}_{\Sigma}(X) \wedge \text{OwnWF}_{\Sigma}(X)\}. \quad (3.13)$$

该空间需要保证良构性  $\text{WF}_{\Sigma}(X)$  和所有权良构性  $\text{OwnWF}_{\Sigma}(X)$ ，前者保证每个活跃的节点都有合法的生成元  $g \in \text{Gen}_{\Sigma}$  标注、只有活跃节点才会被标注、每个活跃边的两个端点都是活跃端点，保证了协调世界的结构完整性；后者保证每个活跃数据token都有活跃节点负责、活跃资源token都有活跃节点持有，每个活跃名字或session都有活跃节点控制。

另外，良构协调状态空间对自己的转移空间是封闭的，数学表示为：

$$X \in \text{St}_{\Sigma} \wedge X \xRightarrow{\mathfrak{e}} Y \implies Y \in \text{St}_{\Sigma}. \quad (3.14)$$

**去端点确定性重放核  $\text{Replay}_{\Sigma, \mathfrak{e}}$**  去端点确定性重放核是一个函数，作为一种计算机制而存在，表示为：

$$\text{Replay}_{\Sigma, \mathfrak{e}} : \text{Recipe}_{\Sigma} \times \text{St}_{\Sigma} \longrightarrow \text{Option}(\text{St}_{\Sigma}). \quad (3.15)$$

反映了由事件配方和源状态何以重计算出目标状态。

**经重放验证的协调事件空间  $\text{Ev}_{\Sigma, \mathfrak{e}}$**  经重放验证的协调事件空间表示，所有完整事件记录中，能够由执行环境的重放核从其源状态重新计算出其目标状态的事件，写作：

$$\text{Ev}_{\Sigma, \mathfrak{e}} = \{e \in \text{DPOEv}_{\Sigma} \mid K_{\mathfrak{e}}(\text{recipe}(e), \text{src}(e)) = \text{some}(\text{tgt}(e))\} \quad (3.16)$$



该空间作为事件集合，包含的事件被表示为  $e \in \text{DPOEv}_\Sigma$ ，即候选事件  $e$  必须首先是一个 DPO 结构重写事件。另外这些事件还需要满足： $K_{\mathfrak{E}}(\text{recipe}(e), \text{src}(e)) = \text{some}(\text{tgt}(e))$ 。

$\text{recipe}(e) = (\rho, m, \kappa, \nu, \alpha)$  是事件配方， $\text{src}(e)$  是输入  $X$  (即事件发生前的状态)；因此  $K_{\mathfrak{E}}(\text{recipe}(e), \text{src}(e))$  表示为在执行环境  $\mathfrak{E}$  中根据事件配方在源状态重新执行一次。但是重新执行不一定能保证完全成功，当重新执行成功时，即表示为  $\text{some}(\text{tgt}(e))$ 。换句话说，用事件配方从源状态重新执行，如果成功，那么得到的结果必须恰好等于事件记录中的目标状态。

**原生合法协调转移关系**  $\Rightarrow_{\Sigma, \mathfrak{E}}$  原生合法协调转移关系也可以表示为  $\Rightarrow_{\Sigma, \mathfrak{E}}^e$ ，表示经过验证的协调事件  $e$  确实以  $X$  为源状态、以  $X'$  为目标状态，构成了一次合法的 原生 协调发生。完整写作：

$$\boxed{X \Rightarrow_{\Sigma, \mathfrak{E}}^e X'} \quad (3.17)$$

它表示协调世界发生了一次合法演化；相比于  $\text{Ev}_{\Sigma, \mathfrak{E}}$  用于说明事件可信，原生合法协调转移关系则表明什么事件真的改变协调世界。

**状态表示等价关系**  $\simeq_{\Sigma, \mathfrak{E}} \subseteq \text{St}_\Sigma \times \text{St}_\Sigma$  状态表示等价关系表示，两个状态虽然使用了不同标识、不同代表元或不同内部表示，但是描述的是同一个可观察的协调世界：

$$\boxed{\simeq_{\Sigma, \mathfrak{E}} \subseteq \text{St}_\Sigma \times \text{St}_\Sigma} \quad (3.18)$$

该定义解决了状态表示和状态语义之间的分离问题，使得系统真正有机会能将语义从“表示”的高度提升到“意义”的高度。

**事件可观察性判定**  $\text{Visible}_{\Sigma, \mathfrak{E}}$  事件可判定观察性判定，决定了哪些原生协调事件进入可观察执行语义，即哪些发生过的事件应该被观察者看到，表示为：

$$\boxed{\text{Visible}_{\Sigma, \mathfrak{E}} : \text{Ev}_{\Sigma, \mathfrak{E}} \longrightarrow \text{Prop.}} \quad (3.19)$$

它接受一个事件  $e$ ，返回一个命题 True/False，判断这个事件是否可观察。

进一步可以定义：

$$\boxed{X \xrightarrow[e]{\text{obs}} X' \iff X \Rightarrow^e X' \wedge \text{Visible}_{\Sigma, \mathfrak{E}}(e).} \quad (3.20)$$

表示事件  $e$  确实发生，且值得被观察。例如，internal 和 external 事件可观察，administrative 事件不可观察。因此行政事件可以合法且可重放，但不进入 observable path。

**派生终态结果分类**  $\text{Outcome}_{\Sigma, \mathfrak{E}}$  派生终态结果分类定义了协调执行最终停在哪里、为什么停止，回应了“当协调系统没有继续可观察动作时意味着什么”这一问题。首先定义：

$$\boxed{\text{Normal}(X) \iff \neg \exists e, X', X \xrightarrow[e]{\text{obs}} X'.} \quad (3.21)$$

表示当不存在  $e$  和  $X'$ , 使  $X \xrightarrow[e]{obs} X'$  时, 即此时已经没有任何可观察事件了, 这种情况表示为  $Normal(X)$ 。

$Normal(X)$  有多种结果, 包括  $SuccessfulTermination$ 、 $ExternalWait$  和  $Deadlocked$  等。将它们的一般形式记为  $Result_i$ , 则 派生终态结果分类可以写作:

$$Outcome : \{X \in St_\Sigma | Normal(X)\} \rightarrow \{Result_i\} \quad (3.22)$$

它们合起来共同构成了协调语义: 一个从良构状态出发, 通过经验证事件不断产生合法后继状态, 并能够对整个执行历史进行重放、等价识别、观察筛选和终态解释的状态转移系统。

进一步的, 从单步语义提升为执行语义, 定义:

$$Path_{\Sigma, \mathfrak{e}}(X, \mathfrak{e}, Y) \quad (3.23)$$

表示事件序列  $e$  将协调世界  $X$  合法演化到  $Y$ 。

则可以递归定义:

$$Path_{\Sigma, \mathfrak{e}}(X, [], X) \quad (3.24)$$

以及:

$$\frac{X \Rightarrow_{\Sigma, \mathfrak{e}}^e Y \quad Path_{\Sigma, \mathfrak{e}}(Y, \mathfrak{e}, Z)}{Path_{\Sigma, \mathfrak{e}}(X, e :: \mathfrak{e}, Z)}. \quad (3.25)$$

因此一段有限的协调过程则表示为:

$$X_0 \xrightarrow{e_1} X_1 \xrightarrow{e_2} X_2 \cdots \xrightarrow{e_n} X_n \quad (3.26)$$

### 3.2.2.2 The Scope of Coordination Semantics / 协调语义的作用范围

在定义协调语义的组成结构后, 还需要明确该语义系统所描述的对象范围。协调语义关注的是: 在给定协调签名与执行环境下, 协调状态如何通过合法事件持续演化, 并形成可验证的执行过程。因此, 其描述对象应限定为由语义规则所诱导的协调执行路径, 即从某一协调状态出发, 经由一系列满足语义约束的事件转移, 最终到达另一协调状态的执行片段。

形式化地, 协调语义的作用范围定义为:

$$Scope(Sem_{\Sigma, \mathfrak{e}}) = \{\tau \mid ObsPath_{\Sigma, \mathfrak{e}}(\tau)\} \quad (3.27)$$

其中, 每条执行路径为:

$$\tau : X_0 \xrightarrow[e_1]{obs} X_1 \xrightarrow[e_2]{obs} \cdots \xrightarrow[e_n]{obs} X_n \quad (3.28)$$

同时满足:



$$\boxed{\forall i < n, \quad X_i \Rightarrow_{\Sigma, \mathfrak{C}}^{e_{i+1}} X_{i+1} \wedge \text{Visible}_{\Sigma, \mathfrak{C}}(e_{i+1})}. \quad (3.29)$$

### 3.3 Four Observational Semantics / 四种观测语义

协调语义完整记录了协调世界的状态、事件及其执行过程，并回答一次状态变化能否合法发生。然而，同一个协调事件通常同时具有多种含义：它可能改变任务依赖关系、转移资源和权限、建立新的通信会话，或者改变一个协调结构与其上下文的组合方式。为了分别揭示这些不同维度，Cantilune 将同一个源协调事件投影到四个独立定义的观测语义中。

#### 3.3.1 General Definition of the Observation Projections / 观测投影的总体定义

Cantilune 用四个观测面构建观测体系：

$$\boxed{I = \{\text{DAG}, \text{Petri}, \pi, \text{Mor}\}} \quad (3.30)$$

其中，DAG 观测协调依赖与因果结构，Petri 观测资源、权限和状态流动， $\pi$  观测通道、命名和会话演化，Mor 观测组合结构与接口关系。

对于任意  $i \in I$ ，定义观测投影：

$$\boxed{P_i : \mathcal{L}_C \rightarrow \mathcal{L}_i} \quad (3.31)$$

其中  $\mathcal{L}_C$  是源协调语义， $\mathcal{L}_i$  是第  $i$  个观测语义。这里每个观测面都不是重新构造一个协调过程，而是对同一个协调过程建立一种解释。

展开观测投影，其内部结构写作：

$$\boxed{P_i = (P_i^S, P_i^E)} \quad (3.32)$$

其中  $P_i^S : S_C \rightarrow S_i$ ，把完整协调状态投影到第  $i$  个观测语义中的状态； $P_i^E : E_C \rightarrow E_i$ ，把完整协调事件投影到第  $i$  个观测语义中的事件。

要实现观测投影，必须满足三个条件：状态等价的良好定义性、正向保持和反向可解释性。

#### 状态等价的良好定义性

状态等价的良好定义性回答：投影后的状态是否仍然代表同一个协调世界。

协调语义中的状态  $X \in S_C$  通常不是唯一表示，例如状态  $X$  和状态  $Y$ ，它们存在差别，例如编号不同、内部字段顺序不同等等，但是并不影响语义的结构调整，因此  $X \approx_C Y$ 。即，两个状态虽然形式不同，但是在协调意义上相同。

该条件保证投影  $P_i^S$  实际作用在  $S_C / \approx_C$  的商空间中。也就是说：

$$\boxed{P_i^S : S_C / \approx_C \rightarrow S_i / \approx_i} \quad (3.33)$$

该式表明：观测的是协调状态本身，而不是状态的某一种编码方式。

**正向保持**

正向保持回答：源协调事件是否真的在目标观测面中发生。它说明每次源协调变化在第  $i$  个观测面中都形成一次原生的可观察变化。

协调语义可以描述出协调事件的发生： $X \xrightarrow[e]{e} Y$ ，而观测语义只关注协调事件中的一部分。因此，正向保持的引入能防止观测语义中的某部分发生遗漏，使得不同角度的观测语义都能同时支持协调事件的发生。

具体来说，正向保持表示为：

$$X \xrightarrow[e]{\text{obs}}_C Y \implies P_i^S(X) \xrightarrow[P_i^E(e)_i]{\text{obs}} P_i^S(Y). \quad (3.34)$$

它表示  $P_i$  是一个迁移保持映射  $P_i : \mathcal{L} \rightarrow \mathcal{L}_i$ ，满足  $\rightarrow_C \subseteq (P_i^S)^{-1} \circ \rightarrow_i \circ P_i^S$ 。简单来说就是，源系统发生的每一步，在目标系统中都有对应的一步。

**反向可解释性**

反向可解释性回答：目标观测面中的变化是否一定来自真实协调事件。它说明从一个投影状态出发，目标语义中发生的任何原生可观察变化，都必须能由某个源协调事件解释。

相比于正向保持回答“源发生的事情，观测面有没有看到”，反向解释则回答“观测面看到的事情，源是否真实发生过”。

具体来说，其表示为：

$$P_i^S(X) \xrightarrow[d]{\text{obs}, i} Z \Rightarrow \exists e, Y : X \xrightarrow[e]{\text{obs}, C} Y \quad (3.35)$$

其中，左侧表示，在第  $i$  个观察面中，如果发生了一次可观察变化；右边表示，那么一定存在源协调事件  $e$ ，使源协调系统也发生了一次对应变化。 $d$  表示触发该变化的目标域事件。

至此，观测投影总体定义完成。观测投影不是把协调状态转换成另一种图形表示，而是在一个独立定义的目标语义中，为源协调状态和事件提供解释，并证明源行为不会丢失、目标行为不会凭空产生、终态判断与版本信息不会发生改变。

**3.3.2 The Four Observation Domains / 四个观测域**

上一节我们对观测语义进行了整体描述，这部分则具体描述，同一个协调事件  $e$ ，在具体哪些语义域中、被解释为什么：

$$X \xrightarrow[e]{\text{obs}}_C Y \mapsto \begin{cases} D_X \xrightarrow[\text{DAG}]{\delta_e} D_Y, \\ M_X \xrightarrow[\text{Petri}]{\Delta_e} M_Y, \\ p_X \xrightarrow[\pi]{\alpha_e} p_Y, \\ f_X \xrightarrow[\text{Mor}]{\chi_e} f_Y. \end{cases} \quad (3.36)$$

### 3.3.2.1 Dependency Structure: DAG Observation Domains / 依赖结构：DAG 观测域

DAG 观测域解释协调事件对依赖结构的影响；其总观测通过 SCC 凝聚保证无环性，而保持原始 incidence 的严格 DAG 只在显式可秩条件下成立。

定义状态对象为：

$$G_X = (V_X, E_X) \quad (3.37)$$

$G_X$  不是完整的协调状态，而是从协调状态中提取出的依赖结构  $G_X = \text{Dep}(X)$ 。其中  $V_X = X.\text{node}$ ，表示协调实体，如 agent、task、artifacts等； $E_X = X.\text{edges}$  表示实体间的关系，如 dependency、prerequisite、causal relation等。

其中要注意的是， $G_X$  中可能包含循环  $u \rightarrow v, v \rightarrow u$ ，甚至包含自环  $u \rightarrow u$ 。因此由于  $\exists G_X, G_X \notin \text{DAG}$ ，因此  $P_{\text{DAG}}$  需要有范围限制。

因此，当协调结构内部存在循环依赖时，DAG 观测域不删除该循环，而是将其压缩为不可分割的依赖组件。这里采用 SCC 压缩：

$$D_X = \text{Cond}(G_X) \quad (3.38)$$

其中：

$$V(D_X) = V(G_X) / \sim_{\text{SCC}} \quad (3.39)$$

此时，一个 DAG 节点就表示一个强连通分量，边  $([u], [v])$  表示存在  $u \rightarrow v$  且  $u \neq v$ 。因此采用 SCC 压缩后图便被压缩为一个有向无环图，此时便可以进行正常投影。

另外，我们对 DAG 可以采取更为严格的定义。为一个图中每个节点分配一个自然数，如果这个图前序节点分配的编号必须要小于后序节点，则该图便是严格的 DAG。用数学则表示为：

$$\text{if } \rho : V_X \rightarrow \mathbb{N}, \text{ s.t. } (u, v) \in E_X \Rightarrow \rho(u) < \rho(v) \quad (3.40)$$

值得注意的是，DAG 观测域并不描述协调事件本身，而是解释该事件如何改变协调对象之间的依赖结构。对于一般有向协调结构，DAG 语义通过强连通分量凝聚获得无环表示；只有在额外满足严格秩约束时，才能保持全部依赖关系而无需折叠。

### 3.3.2.2 Resource Flow: Petri Observation Domains / 资源流：Petri 观测域

Petri 观测域用于描述一次协调变化需要、保留、消耗和产生了哪些具有稳定身份的资源或证据。数学上定义为：

$$M_X = \text{Prov}(X) \quad (3.41)$$

该定义没有把 marking 简化为匿名 token 数量，而是定义具有 provenance 的个体 token，包括签名版本、节点边及节点标签、数据 token 与其 owner、资源 token 与其 owner、名称与 session owner、外部观察、policy 状态等。

Petri 事件由协调事件发生前后对应协调状态的精确差分决定，数学表示为：

$$\text{For } X \xrightarrow[e]{\text{obs}} Y, \Delta_e^- = M_X \setminus M_Y, \quad \Delta_e^+ = M_Y \setminus M_X \quad (3.42)$$

合起来即为：

$$\Delta_e = (\Delta_e^-, \Delta_e^+) \quad (3.43)$$

因此，一个 Petri 事件被触发（fire）后，token 如何从一个 marking（状态）变化到另一个 marking（状态）的过程可以表示为：

$$(M_X - \Delta_e^-) \cup \Delta_e^+ = M_Y \quad (3.44)$$

另外当 token 同时存在于事件触发前后的协调状态时，则它既不属于正差分、也不属于负差分，数学表示为：

$$a \in M_X \cap M_Y \implies a \notin \Delta_e^- \wedge a \notin \Delta_e^+ \quad (3.45)$$

同时，Petri 域还包含有序规则声明：

$$d = (\nu, \text{ruleId}, k) \quad (3.46)$$

其中  $\nu$  是版本， $k$  是声明序号； $(\nu, \text{ruleId})$  唯一标识一项规则。在签名准入时，新声明只能追加，旧声明及其 incidence 必须经重索引继续保留。

### 3.3.2.3 Communication and Name Passing: $\pi$ Observation Domains / 通信与名称传递: $\pi$ 观测域

$\pi$  观测域负责观测协调变化在 agent 交互层面具体是哪一次通信、同步、名称传递或作用域变化，专门描述 agent 交互层面的通信行为。

在这里，我们按照  $\pi$ -calculus 的表示习惯，将协调状态  $X$  重写为  $p_X$ ，表示为“一个状态就是一个进程 (Process)”。针对事件中的通信过程，利用  $\pi$ -calculus 将其表示为：

$$\alpha \in \{\tau, \bar{a}(b), a(x), \bar{a}(b)\} \quad (3.47)$$

其中  $\tau$  表示内部通信完成， $\bar{a}(b)$  表示通过 channel  $a$  发送信号  $b$ ， $a(x)$  表示等待接收、将收到的来自 channel  $a$  的信号绑定至变量  $x$ ， $\bar{a}(b)$  表示发送一个新的私有名字。

在  $\pi$  观测域下，协调过程被投影为通信过程，即：

$$P_\pi^E(e) = (e, \alpha_e) \quad (3.48)$$

不只保留  $\alpha_e$ 。是因为两个不同的协调事件可能都表现为  $\tau$ ，为了防止动作标签可能丢失事件身份，因此也将源事件纳入到投影中。

因此，一次源协调变化应解释为一个原生强迁移：

$$p_X \xrightarrow{\alpha_e}_{\pi} p_Y. \quad (3.49)$$

进一步的，由于 Cantilune 不希望任意通信，协调变化进一步被表示为：

$$p_X \xrightarrow{\alpha_e}_{typed\ \pi} p_Y \quad (3.50)$$

应用擦除类型  $|\cdot|$  后，又可以得到普通  $\pi$ ，即：

$$|p_X| \xrightarrow{|\alpha_e|} |p_Y| \quad (3.51)$$

即去掉 Cantilune 额外约束后，它仍然是标准  $\pi$ -calculus 合法步骤。

$\pi$  观测域把协调变化解释为 agent 之间真实发生的一次强通信步骤，并显式记录同步、名称传递、绑定和作用域变化。

#### 3.3.2.4 Compositional Structure: Morphism Observation Domains / 组合结构：态射观测域

态射观测域反映协调变化如何作为一个保持类型接口和组合结构的态射变化出现。

在投影时，态射观测域不主动丢失信息：

$$P_{Mor}^S(X) = X, P_{Mor}^E(e) = e, \quad (3.52)$$

因此  $X \xrightarrow[e]{obs} Y$  被看作：

$$f_X \xrightarrow{\chi_e} f_Y \quad (3.53)$$

在 Cantilune 中，态射  $f_X$  表示状态  $X$  对应的结构， $\chi_e$  在结构层面的变换。在完整的范畴层中，每个 LTS 状态被实现为箭头：

$$f_X = \text{stateArrow}(X), \quad f_Y = \text{stateArrow}(Y), \quad (3.54)$$

而一次可观察步骤被实现为箭头范畴中的态射：

$$\chi_e : f_X \longrightarrow f_Y. \quad (3.55)$$

静态函子  $F$  与操作投影之间必须满足交换条件：

$$F^{\rightarrow}(\chi_e) = \iota_X \gg \chi_{P(e)} \gg \iota_Y^{-1}. \quad (3.56)$$

这说明静态协调结构经过函子解释得到的重写，与操作 LTS 中实际发生的目标迁移是同一个交换方块，而不只是事件名称相似。因此综合来看，态射观测域保留协调变化的类型接口与组合形式，并把一次运行时迁移解释为两个结构箭头之间的可组合变换。

综上所述，四个观测域对同一次协调发生的四种互补解释：DAG 域说明依赖结构如何改变，Petri 域说明具有稳定身份和归属的资源如何流动， $\pi$  域说明 agent 之间实际发生了什么通信与名称传递，态射域则说明这次变化如何保持类型接口和组合结构。四者共同回指同一个源状态  $X$ 、源事件  $e$  和后继状态  $Y$ 。相应的投影证书进一步保证，每种解释都是其目标语义中的原生变化，并且从投影像出发的目标变化都能够反向追溯到源协调语义。该过程完整写作：

$$X \xrightarrow[e]{\text{obs}} Y \mapsto \left( D_X \xrightarrow[\text{DAG}]{\delta_e} D_Y, M_X \xrightarrow[\text{Petri}]{\Delta_e} M_Y, p_X \xrightarrow[\pi]{\alpha_e} p_Y, f_X \xrightarrow[\text{Mor}]{\chi_e} f_Y \right). \quad (3.57)$$

至此，四观测域把一次抽象的“合法协调变化”分解为四个可分别验证的问题：

$$\boxed{\text{结构依赖} + \text{资源流动} + \text{交互行为} + \text{组合形式}.} \quad (3.58)$$

这四个方面共同构成 Cantilune 对同一协调生命过程的多维语义解释。

### 3.4 Coherence of Coordination Across Observation Domains and Epochs / 跨观测域与 Epoch 的协调一致性

在上一章，协调语义被拆解为四个观测语义描述。本章则说明，这些不同视角、不同时间、不同版本下的解释，仍然可以被认为描述的是同一个协调过程。

#### 3.4.1 Execution-Level Coherence Across Observation Views / 观测视角下的执行一致性

前一章证明，在发生单次协调事件时，该协调过程可以通过四个角度进行观测，但是并未证明在事件序列中，如果源协调系统完成了一段生命周期，那么任何观测域能否重建该生命周期。

我们首先定义执行轨迹  $\text{Path}_\Sigma(X, \mathbf{e}, Y)$  表示存在事件序列  $\mathbf{e} = [e_1, \dots, e_n]$  使得：

$$\boxed{X = X_0 \xrightarrow[e_1]{\text{obs}} X_1 \dots \xrightarrow[e_n]{\text{obs}} X_n = Y} \quad (3.59)$$

然后，我们分别对不同观测域，引入路径正向保持、路径反射和终态一致三个性质。路径正向保持保证局部事件解释一致，路径一致性保证完整协调生命周期一致，终态一致性保证不同观测域对生命周期结果形成统一判断。

**路径正向保持** 基于路径长度归纳，由单步投影保持性，可以推导出路径正向保持性质：

$$\boxed{\text{Path}_C(X, \mathbf{e}, Y) \Rightarrow \text{Path}_i(P_i^S(X), P_i^E(\mathbf{e}), P_i^S(Y))} \quad (3.60)$$

其中  $P_i^E(\mathbf{e}) = [P_i^E(e_1), \dots, P_i^E(e_n)]$ 。路径正向保持定理表明，如果源协调系统完成了一段生命周期，那么任何观测域都能够重建该生命周期。

**路径反射** 为了避免目标域可能出现  $P_i(X) \rightarrow Z$  但是源系统不存在对应变化的情况，通过目标路径结构归纳，并逐步应用单步 reflection，可以得到路径反射性质：

$$\boxed{\text{Path}_i(P_i^S(X), \mathbf{d}, Z) \Rightarrow \exists \mathbf{e}, Y} \quad (3.61)$$

满足  $\text{Path}_C(X, \mathbf{e}, Y)$ ，且  $\forall k, \text{Lift}_i(e_k, d_k)$ ，同时  $Z \approx_i P_i^S(Y)$ 。该性质防止观测域产生源系统不存在的虚假执行。任意从投影状态出发的目标轨迹，都必须能够提升为源协调语义中的真实执行。

**终态一致** 源协调系统完成了一段生命周期则会进入终态，因此定义  $\text{Normal}(X)$ ，表示  $\nexists e, Y, X \xrightarrow[e]{\text{obs}} Y$ 。因为由于路径 soundness 保证源执行不会在目标域中消失，而路径 reflection 保证目标执行不会凭空产生，因此二者共同推出终态结构保持性质：

$$\boxed{\text{Normal}(X) \iff \text{Normal}_i(P_i(X))} \quad (3.62)$$

终态一致性保证了不同观测域对于生命周期结果给出相同判断。

### 3.4.2 Semantic Preservation Under Signature Evolution / 签名演化下的语义保持

在前面的描述中， $\Sigma$  总是被固定，本节讨论当有新的语义变化加入时旧世界还能不能保持原来的意义。

首先定义：

$$\boxed{\iota : \Sigma \hookrightarrow \Sigma'} \quad (3.63)$$

进一步定义状态重索引，对协调状态和协调事件在旧语言  $\Sigma$  向新语言  $\Sigma'$  转换时发生的改变进行描述：

$$\boxed{\text{reindex}_C^\iota : St_\Sigma \rightarrow St_{\Sigma'}, \text{reindex}_C^\iota : Ev_\Sigma \rightarrow Ev_{\Sigma'}} \quad (3.64)$$

进一步经证明得出：

$$\boxed{P_{i,\Sigma'}^S(\text{reindex}_C^\iota(X)) = \text{reindex}_i^\iota(P_{i,\Sigma}^S(X)), P_{i,\Sigma'}^E(\text{reindex}_C^\iota(e)) = \text{reindex}_i^\iota(P_{i,\Sigma}^E(e))} \quad (3.65)$$

至此我们可以得到，当新规则加入系统，旧状态会自动适配进新状态、因而旧世界还能保持原来的意义。而新规则本身又可以作为协调事件的一部分，参与协调状态的更新：

$$\boxed{X^- \xrightarrow[a]{\text{admission}} X^+} \quad (3.66)$$

由此可得，当 Agent Orchestration 的规则空间自身发生变化时，旧的协调历史可以继续保持语义连续，新规则又可以作为新的协调事件被引入，从而实现了语义保持。

### 3.4.3 Occurrence Identity Across Epoch Transitions / Epoch 转换中的发生身份保持

这一部分讨论，同一个协调过程，在不同表示和不同时间尺度下是否保持同一语义。

考虑这样一种场景， $\Sigma_0 \hookrightarrow \Sigma_1$  并发生  $X^- \xrightarrow[a]{\text{admit}} X^+$  随后  $X^+ \xrightarrow[e]{\text{obs}} Y$ ，这里  $a$  是版本变化事件、 $e$  是后续业务事件。

那么现在的问题是，该场景中在不同角度观测下， $P_i(a)$  和  $P_i(e)$  是否仍然对应同一条源协调历史  $(a, e)$ 。



因此定义四域跨 epoch 一致性：

$$\text{CrossEpochAgree}(\mathbf{P}, a, e). \quad (3.67)$$

它表示固定签名下的路径一致性、投影自然性、原生准入步骤和 occurrence identity 被统一到同一个证明对象中。

其中包含 Replay 连续性，要求  $\text{Replay}_{\Sigma_0}(e, X) = Y$ ，则  $\text{Replay}_{\Sigma_1}(\text{reindex}(e), \text{reindex}(X)) = \text{reindex}(Y)$ ；Projection 连续性，对于每个观测域有  $P_{i, \Sigma_1}(\text{reindex}(e)) = \text{reindex}_i(P_{i, \Sigma_0}(e))$ ；Occurrence Identity 连续性，定义事件身份  $\text{id}(e)$ ，要求  $\text{id}_{\Sigma_0}(e) = \text{id}_{\Sigma_1}(\text{reindex}(e))$ ，即同一个协调发生不会因为语言升级而失去身份。

综上所述，结合观测视角下的执行一致性、签名演化下的语义保持和转换中的发生身份保持，可以得出，一个协调发生可以在多个观察空间、多个语言版本和多个执行阶段中保持统一的语义身份。

### 3.5 Feedback Stabilization and Core Theoretical Closure / 反馈稳定化与核心理论闭包

前面的理论依次回答了四个问题：协调世界由什么构成，协调事件如何合法发生，同一次协调发生在四个观测域中分别意味着什么，以及这些解释如何跨路径与 epoch 保持一致。本节讨论最后一个问题：当协调系统持续接收新的证据并据此演化时，这一过程能否最终进入稳定状态。

为此，Cantilune 将反馈过程建模为有限高度证据序上的单调演化，并通过一个由原生协调事件支撑的有限 Markov kernel 描述不同合法后继之间的随机选择。在稳定签名窗口、公平观测机会和统一正进展概率成立时，系统几乎必然进入一个向上闭的稳定证据区域；进入该区域以后，证据单调性保证系统不会再次退出。

最后，本节将协调语义、四观测域、跨 epoch 一致性与反馈稳定化组合起来，给出本章完整的数学闭包。

#### 3.5.1 Feedback Dynamics and Almost-Sure Stabilization / 反馈动力学与几乎必然稳定化

**有限高度证据序** 协调系统在执行过程中会不断获得新的观察、审核结果、外部决策和运行证据。为了描述这些信息如何累积，定义证据空间  $Q$  为一个具有 join 运算的半格：

$$(Q, \sqsubseteq, \sqcup) \quad (3.68)$$

其中， $q \sqsubseteq q'$  表示  $q'$  至少包含  $q$  所提供的信息，而  $q \sqcup q'$  表示将两份证据合并。证据更新只能通过 join 增加信息，不能删除已经获得的证据。

为了排除无限严格增长，进一步给出有限高度  $H \in \mathbb{N}$  和秩函数

$$h : Q \longrightarrow \mathbb{N} \quad (3.69)$$

满足：

$$\forall q \in Q, \quad h(q) \leq H, \quad q \sqsubset q' \implies h(q) < h(q'). \quad (3.70)$$

因此，若

$$q_0 \sqsubset q_1 \sqsubset \cdots \sqsubset q_m \quad (3.71)$$

是一条严格证据增长链，则必有：

$$h(q_0) + m \leq H. \quad (3.72)$$

这说明证据可以保持不变，也可以有限次严格增长，但不可能沿证据序无限严格上升。

**稳定证据区域** 定义稳定证据区域  $U \subseteq Q$  为一个向上闭集：

$$q \in U \wedge q \sqsubseteq q' \implies q' \in U. \quad (3.73)$$

$U$  表示已经满足稳定性要求的证据状态集合。向上闭性说明：如果现有证据已经足以判定系统稳定，那么继续获得更多证据不会使该稳定性失效。

令

$$\eta : \text{St}_\Sigma \longrightarrow Q \quad (3.74)$$

为协调状态的证据解释函数，其中  $\eta(X)$  表示协调状态  $X$  当前携带的累积证据。

要求每一次原生可观察协调变化都保持证据单调性：

$$X \xrightarrow[e]{\text{obs}} Y \implies \eta(X) \sqsubseteq \eta(Y). \quad (3.75)$$

由式 (3.73) 和式 (3.75) 可以直接得到反馈系统的确定性稳定保持性质：

$$\begin{aligned} & \text{Path}_{\Sigma, \mathfrak{e}}(X_0, [e_1, \dots, e_n], X_n) \wedge \eta(X_0) \in U \\ & \implies \forall k \leq n, \quad \eta(X_k) \in U. \end{aligned} \quad (3.76)$$

因此，稳定区域  $U$  是协调执行的前向不变量。该结论不依赖任何概率假设：只要一次执行已经进入  $U$ ，后续任意有限原生执行都不会使其退出。

**内部执行的有限性** 反馈稳定化还需要排除系统在同一个 epoch 内无限进行内部管理动作。为此，对内部事件给出自然数值秩

$$\rho_{\text{int}} : \text{St}_\Sigma \longrightarrow \mathbb{N}. \quad (3.77)$$

它是一个证明用的度量，表示“当前 epoch 内还允许/还可能剩多少内部整理动作”。

进一步的，要求每个内部事件严格降低该秩，同时保持所在 epoch 不变。于是，对于完全由内部事件构成的路径

$$\text{Path}_{\Sigma, \mathfrak{e}}(X, \mathbf{e}, Y). \quad (3.78)$$

有：

$$\mathbf{e} \text{ 中的事件全部为内部事件} \implies \begin{cases} |\mathbf{e}| \leq \rho_{\text{int}}(X), \\ \text{epoch}(Y) = \text{epoch}(X). \end{cases} \quad (3.79)$$

因而不存在一条完全由内部事件组成的无限执行。系统不能通过无限行政或内部振荡永久回避后续的可观察反馈机会。

**由原生事件支撑的有限 Markov kernel** 证据单调性只能证明“进入稳定区域后不会退出”，不能单独证明系统一定会进入稳定区域。为了描述不同合法后继之间的随机选择，令  $S_K$  为有限的 kernel 状态空间，并给出与源协调执行状态之间的等价：

$$\varphi : S_K \simeq \text{St}_\Sigma. \quad (3.80)$$

在这里， $\varphi$  是一个双射，将 kernel 展现为真实协调状态的一个等价编码上的随机游走。

定义有限 Markov kernel：

$$K : S_K \times S_K \longrightarrow [0, 1] \quad (3.81)$$

表示从状态  $s$  转移到状态  $t$  的概率。

满足：

$$\forall s, t \in S_K, \quad K(s, t) \geq 0, \quad \forall s \in S_K, \quad \sum_{t \in S_K} K(s, t) = 1. \quad (3.82)$$

概率矩阵本身只描述状态选择。为了保证随机轨迹仍然是一条真实的协调执行轨迹，还必须为每条正概率边提供原生事件标签。也就是说，当  $K(s, t) > 0$  时，存在事件  $e_{s,t}$ ，满足：

$$K(s, t) > 0 \implies \begin{cases} \varphi(s) \xrightarrow[e_{s,t}]{\text{obs}} \varphi(t), \\ \text{Replay}_{\Sigma, \mathfrak{C}}(\text{recipe}(e_{s,t}), \varphi(s)) = \text{some}(\varphi(t)). \end{cases} \quad (3.83)$$

因此，kernel 不会通过正概率边引入源协调语义中不存在的状态变化。被随机选择的每次真实变化都携带同一个原生协调事件及其精确 replay 见证。反馈稳定化不是只研究某一条确定执行路径，而是允许系统在多个合法后继之间随机选择；但每一次正概率选择都必须对应真实协调语义中的原生事件，并且可以被 replay 精确验证。因此，随机轨迹是一条由真实协调事件装饰起来的协调执行轨迹。

由初始分布  $\mu_0$  和 kernel  $K$  可以构造无限状态轨迹上的概率测度：

$$\mathbb{P}_{K, \mu_0} \quad \text{on} \quad S_K^{\mathbb{N}}. \quad (3.84)$$

在该测度下，几乎每条采样路径的相邻状态都由正概率边连接。由式 (3.83)，这些边可以被统一装饰为原生、可重放的协调事件轨迹。

**稳定窗口与公平观测机会** 概率进展结论只在签名保持稳定的执行后缀中成立。定义稳定公平窗口

$$\mathcal{W} = (\nu, \text{Observed}, n_0, \tau) \quad (3.85)$$

其中：

1.  $\nu : \mathbb{N} \rightarrow \mathbb{N}$  给出每个 epoch 的签名版本；
2.  $n_0$  是稳定窗口的起始 epoch；
3.  $\tau(j)$  是第  $j$  个合格观测机会实际发生的 epoch；
4.  $\text{Observed}(n)$  表示 epoch  $n$  包含一个合格观测机会。

稳定公平窗口需要满足：

$$\begin{aligned} &\forall k \in \mathbb{N}, \quad \nu(n_0 + k) = \nu(n_0), \\ &\forall j \in \mathbb{N}, \quad n_0 \leq \tau(j), \\ &j < j' \implies \tau(j) < \tau(j'), \\ &\forall j \in \mathbb{N}, \quad \text{Observed}(\tau(j)), \\ &\forall n \geq n_0, \quad \exists j, n \leq \tau(j). \end{aligned} \quad (3.86)$$

第一项保证窗口从  $n_0$  开始后签名版本不再变化；第二项保证所有被计数的观测机会都发生在稳定窗口开始后；第三项表示按时间严格向前，不会重复计算同一个 epoch；第四项保证  $\tau(j)$  指到的都是合格的观测机会；第五项保证未来不会停止出现观测机会。对任意晚的 epoch  $n$ ，总还能找到一个不早于  $n$  的观测机会。

如果还要求每个窗口后 epoch 都恰好对应一个观测机会，则得到更强的逐 epoch 公平性：

$$\forall j \in \mathbb{N}, \quad \tau(j) = n_0 + j. \quad (3.87)$$

一般公平性足以说明合格机会持续发生；只有式 (3.87) 成立时，按“机会数量”得到的数值界才能同时被解释为按“窗口后 epoch 数量”得到的界。

**阶段进展与统一正概率下界** 该部分证明只要每次合格观测机会都有至少  $\varepsilon$  的概率完成当前阶段，那么当前阶段几乎必然会在有限次机会内完成。

有限高度  $H$  表示最多只需考虑  $H$  个严格证据增长阶段，系统从“不稳定”到“稳定”最多需要经历  $H$  个严格证据进展阶段，不会有无限多层证据升级。对每个阶段

$$r \in \{0, \dots, H-1\}. \quad (3.88)$$

给出初始分布  $\mu_r$  和阶段稳定谓词

$$b_r : S_K \longrightarrow \{0, 1\}. \quad (3.89)$$

其中  $b_r(s) = 1$  表示该阶段要求的下一次证据进展已经完成， $b_r(s) = 0$  则表示未完成。选定一个特殊阶段  $r_*$ ，使其稳定谓词恰好对应证据稳定区域  $U$ ，即正好证据已经进入稳定区域  $U$ 。：

$$b_{r_*}(s) = 1 \iff \eta(\varphi(s)) \in U. \quad (3.90)$$

假设存在统一常数  $\varepsilon$ ，满足

$$0 < \varepsilon \leq 1, \quad (3.91)$$

并且在任意尚未完成阶段  $r$  的状态中，下一次 kernel 转移有至少  $\varepsilon$  的概率直接进入该阶段的完成集合：

$$b_r(s) = 0 \implies \sum_{\substack{t \in S_K \\ b_r(t)=1}} K(s, t) \geq \varepsilon. \quad (3.92)$$

令  $m_r(n)$  表示在阶段  $r$  中，经过前  $n$  个合格观测机会后仍未命中  $b_r = 1$  的概率。由式 (3.92) 可以从同一个 kernel 推导出：

$$m_r(n+1) \leq (1 - \varepsilon)m_r(n). \quad (3.93)$$

递归应用该式可得几何尾界：

$$m_r(n) \leq (1 - \varepsilon)^n. \quad (3.94)$$

由于  $0 < \varepsilon \leq 1$ ，有

$$\lim_{n \rightarrow \infty} (1 - \varepsilon)^n = 0. \quad (3.95)$$

这说明：随着合格观测机会次数增加，“一直没完成”的概率趋向于 0。因此，每个阶段永不命中的轨迹集合具有零测度：

$$\mathbb{P}_{K, \mu_r}(\forall n, b_r(\omega_n) = 0) = 0. \quad (3.96)$$

等价地：

$$\mathbb{P}_{K, \mu_r}(\exists n, b_r(\omega_n) = 1) = 1. \quad (3.97)$$

阶段  $r$  以概率 1 会在某个有限时刻完成。即在有限状态 Markov kernel 中，如果每个未完成状态在每次合格机会都有统一下界  $\varepsilon$  的概率进入完成集合，那么未完成概率按几何速度衰减，因此该阶段几乎必然完成。

**几乎必然永久稳定** 对特殊阶段  $r_*$ ，式 (3.97) 首先说明轨迹几乎必然在有限时间内命中证据稳定区域  $U$ 。随后，由证据单调性和  $U$  的向上闭性，命中不再只是一次暂时访问，而会成为永久保持。

定义：

$$\text{EventuallyAlways}_U(\omega) \iff \exists N \in \mathbb{N}, \forall n \geq N, \eta(\varphi(\omega_n)) \in U. \quad (3.98)$$

于是得到：

$$\mathbb{P}_{K, \mu_{r*}} (\text{EventuallyAlways}_U) = 1. \quad (3.99)$$

该结论由两个不同性质组合而成：正进展概率保证系统几乎必然在有限时间内进入  $U$ ，而确定性前向不变量保证系统一旦进入  $U$  就永远不会退出。

**有限高度期望界** 对每个阶段，将 miss probability 的尾和定义为该阶段的期望等待量。全部  $H$  个阶段的总等待量定义为：

$$\text{Wait}_{H, \varepsilon} := \sum_{r=0}^{H-1} \sum_{n=0}^{\infty} m_r(n). \quad (3.100)$$

由几何尾界可得：

$$\text{Wait}_{H, \varepsilon} \leq \frac{H}{\varepsilon}. \quad (3.101)$$

该式在一般情况下首先是合格观测机会数量的 tail-sum 界。当逐 epoch 公平性 (3.87) 成立时，每个窗口后 epoch 恰好提供一次机会，因此它可以进一步解释为完成全部剩余严格证据增长阶段所需的期望 epoch 数上界。

#### Almost-sure stabilization / 几乎必然稳定化

在有限高度证据序、向上闭稳定区域、原生事件支撑的有限 Markov kernel、稳定签名窗口、逐 epoch 公平性和统一正进展概率  $0 < \varepsilon \leq 1$  下：

$$\mathbb{P}_{K, \mu_{r*}} (\text{EventuallyAlways}_U) = 1, \quad \text{Wait}_{H, \varepsilon} \leq \frac{H}{\varepsilon}. \quad (3.102)$$

几乎每条被采样的正概率路径同时携带原生协调事件、精确 replay 和稳定窗口中的 epoch 对齐见证。

这里的概率 1 不表示每一条数学上可能的轨迹都必然稳定，而表示不稳定轨迹集合在由  $K$  和初始分布生成的轨迹测度下具有零测度。同样，若签名不断变化、观测机会停止出现，或者不存在统一的正进展下界  $\varepsilon$ ，则上述活性结论不成立。

### 3.5.2 The Complete Core Theorem and Its Scope / 完整核心定理及其成立范围

前面的各部分并不是相互独立的数学模型。协调语义提供唯一的源执行过程，四个观测域解释同一过程的不同侧面，跨 epoch 一致性保证这些解释在签名演化后仍然指向同一条协调历史，而反馈语义则在同一原生执行空间上描述该历史的长期演化。

因此，完整核心定理不再引入新的协调对象，而是将已经建立的语义与证书组合为一个共同结论。

令

$$\mathbf{P} = (P_i)_{i \in I}, \quad I = \{\text{DAG}, \text{Petri}, \pi, \text{Mor}\}. \quad (3.103)$$

对每个  $i \in I$ , 记

$$\begin{aligned} \text{CompleteProj}(P_i) := & \text{WellDefined}(P_i) \\ & \wedge \text{StepSound}(P_i) \wedge \text{StepReflect}(P_i) \\ & \wedge \text{PathSound}(P_i) \wedge \text{PathReflect}(P_i) \\ & \wedge \text{TermCons}(P_i) \wedge \text{VersionCons}(P_i). \end{aligned} \quad (3.104)$$

该条件分别保证：投影不依赖源状态的表示方式，源步骤不会在目标域中消失，目标步骤不会凭空出现，有限执行路径可以双向解释，并且终态分类与签名版本不会因投影发生改变。

考虑签名嵌入

$$\iota : \Sigma \hookrightarrow \Sigma' \quad (3.105)$$

以及一次原生准入事件

$$X^- \xrightarrow[a]{\text{admission}} X^+. \quad (3.106)$$

定义 epoch 一致性前提：

$$\begin{aligned} \text{EpochCert}(\mathbf{P}, \iota, a, e) := & \text{ProjectionNaturality}(\mathbf{P}, \iota) \\ & \wedge \text{ReplayNaturality}(\iota) \\ & \wedge \text{NativeAdmission}(a) \\ & \wedge \text{OccurrencePreservation}(\iota, a, e). \end{aligned} \quad (3.107)$$

其中，投影自然性保证先重索引再投影与先投影再重索引得到相同结果；replay 自然性保证旧事件在新签名中仍然重放到重索引后的目标；原生准入保证版本变化本身是一次真实协调发生；发生身份保持则保证事件不会因为跨越 epoch 而被错误地替换为另一个 occurrence。

最后，以

$$\text{FeedbackCert}(Q, U, \eta, K, \mathcal{W}, H, \varepsilon) \quad (3.108)$$

表示上一节给出的完整反馈数据及其性质，包括有限高度证据序、向上闭稳定区域、原生证据单调性、有限原生 kernel、正概率事件标记、稳定公平窗口、阶段进展桥和  $0 < \varepsilon \leq 1$ 。

在这些条件下，Cantilune 的完整核心定理可以表述为：

#### Complete core theorem / 完整核心定理

若

$$\forall i \in I, \quad \text{CompleteProj}(P_i), \quad (3.109)$$

且

$$\text{EpochCert}(\mathbf{P}, \iota, a, e) \wedge \text{FeedbackCert}(Q, U, \eta, K, \mathcal{W}, H, \varepsilon) \quad (3.110)$$



成立，则存在共同一致性见证  $W$ ，使得：

$$\begin{aligned}
 & \text{FourViewAgree}(\mathbf{P}; W) \\
 & \wedge \text{CrossEpochAgree}(\mathbf{P}, \iota, a, e; W) \\
 & \wedge \text{ReplayAligned}(K, W) \\
 & \wedge \mathbb{P}_{K, \mu_{r*}}(\text{EventuallyAlways}_U) = 1 \\
 & \wedge \text{Wait}_{H, \varepsilon} \leq \frac{H}{\varepsilon}.
 \end{aligned} \tag{3.111}$$

其中， $\text{FourViewAgree}(\mathbf{P}; W)$  表示同一条源协调执行在 DAG、Petri、 $\pi$  和态射四个观测域中具有相容的状态、事件、路径和终态解释； $\text{CrossEpochAgree}(\mathbf{P}, \iota, a, e; W)$  表示这些解释在签名重索引、准入事件和后续业务事件之间保持 replay、投影和 occurrence identity 连续； $\text{ReplayAligned}(K, W)$  则表示随机轨迹所选择的正概率边使用的正是源协调语义中的原生事件，并且这些事件与其 replay 端点和执行 epoch 对齐。

因此，本章的完整数学推导可以概括为：

$$\begin{aligned}
 & \text{协调状态、协调事件与规则空间} \\
 & \implies \text{合法、可观察且可重放的协调执行} \\
 & \implies \text{DAG、Petri、}\pi \text{ 与态射四域中的原生解释} \\
 & \implies \text{路径、终态、版本与 occurrence 的跨域一致性} \\
 & \implies \text{签名演化与准入过程中的语义连续性} \\
 & \implies \text{由同一原生执行支撑的反馈随机轨迹} \\
 & \implies \text{几乎必然进入并永久保持在稳定证据区域.}
 \end{aligned} \tag{3.112}$$

这条推导链形成了本章的理论闭环：协调世界给出被改变的对象，协调语义给出变化发生的条件，四观测域给出变化的多维含义，一致性定理保证这些含义仍然属于同一个协调发生，反馈稳定化则说明该执行过程在明确概率前提下最终会进入何种长期状态。

**定理的成立范围** 完整核心定理是一个条件定理，其结论严格受以下范围限制：

1. 四域一致性以完整投影证书为前提。定理组合并检查已经给出的投影证书，但不会为任意未经认证的 Agent Orchestration 自动构造这些证书。
2. 跨 epoch 结论要求显式签名嵌入和原生准入事件。单纯对旧对象进行重索引不能代替一次真正改变签名版本的准入发生。
3. 随机稳定化以有限 kernel 状态空间和有限高度证据序为前提，不直接覆盖无限状态 kernel 或无限严格证据增长链。
4. 几乎必然命中要求签名在某个后缀中保持稳定、合格观测机会持续出现，并且每个未完成阶段具有统一正进展下界  $\varepsilon$ 。
5.  $H/\varepsilon$  首先是全部有限高度阶段的 tail-sum opportunity 界。只有在逐 epoch 公平性成立时，它才具有窗口后 epoch 数量的解释。
6. 概率 1 允许存在零测度的不稳定轨迹，因此不能被解释为“所有轨迹无例外地在有限时间内稳定”。
7. 本节闭合的是协调执行、四观测域、跨 epoch 身份与反馈稳定化之间的核心理论。对  $\pi$  过程与更进一步指称模型之间的精确语义缝合属于后续理论扩展，不作为本节核心闭包的前提或结论。

最后，项目中的有限参考模型为上述各类证书提供了非空见证，并包含真实改变协调结构的原生事件。因此，这一理论并非仅由恒等投影或空执行满足。但是，参考模型的非空性只证明理论条件可以被真实模型满足，并不等同于所有生产环境已经自动获得相同的完备性、公平性或稳定化保证。

# References / 参考文献

---

- Samson Abramsky and Achim Jung. Domain theory. In Samson Abramsky, Dov M. Gabbay, and T. S. E. Maibaum, editors, *Handbook of Logic in Computer Science*, volume 3, pages 1–168. Clarendon Press, Oxford, 1994.
- Roberto Bruni, Jose Meseguer, Ugo Montanari, and Vladimiro Sassone. Functorial models for petri nets. *Information and Computation*, 170(2):207–236, 2001. doi: 10.1006/inco.2001.3050.
- Marcelo P. Fiore, Eugenio Moggi, and Davide Sangiorgi. A fully abstract model for the pi-calculus. *Information and Computation*, 179(1):76–117, 2002. doi: 10.1006/inco.2002.2968.
- Stephen Lack and Paweł Sobociński. Adhesive categories. In *Foundations of Software Science and Computation Structures*, volume 2987 of *Lecture Notes in Computer Science*, pages 273–288. Springer, 2004. doi: 10.1007/978-3-540-24727-2\_20.
- Stephen Lack and Paweł Sobociński. Adhesive and quasiadhesive categories. *RAIRO – Theoretical Informatics and Applications*, 39(3):511–545, 2005. doi: 10.1051/ita:2005028.
- Saunders Mac Lane. *Categories for the Working Mathematician*, volume 5 of *Graduate Texts in Mathematics*. Springer, New York, 2 edition, 1998. doi: 10.1007/978-1-4757-4721-8.
- Moonweave AI. Cantilune core theory closure delivery. Theory-closure technical-delivery record, 2026a. Updated 2026-07-28; accessed 2026-07-28.
- Moonweave AI. Cantilune formal semantics: Lean project and evidence gate. Formal-project README and evidence-gate guide, 2026b. Accessed 2026-07-28.
- Moonweave AI. Cantilune proof-obligation manifest. Machine-readable proof manifest, 2026c. Accessed 2026-07-28.
- Moonweave AI. Rfc-0001: Cantilune architecture. Architecture RFC, 2026d. Accessed 2026-07-28.
- Moonweave AI. Rfc-0002: Projection consistency — what the proof buys, and the four-projection consistency theorem. Architecture and formal RFC, 2026e. Updated 2026-07-28; accessed 2026-07-28.
- Moonweave AI. Cantilune: Compose intent into visible, evolvable coordination. Repository README on the reviewed branch, 2026f. Accessed 2026-07-28.
- Moonweave AI. Cantilunegraph v0.1 — formal semantics specification. Normative formal-semantics specification, 2026g. Updated 2026-07-28; accessed 2026-07-28.